CMSC132: File I/O

Chapter 18

By Naren Ayinala

May 2026

Contents

1 What Exactly Is File I/O? 3

2 Files, Paths, and Streams 3

2.1 Relative and Absolute Paths . . . 3

2.2 The File and Path Classes . . . 3

3 Text Files 4

3.1 Writing to Text Files . . . 4

3.1.1 FileWriter . . . 4

3.1.2 BufferedWriter . . . 5

3.1.3 PrintWriter . . . 5

3.2 Reading from Text Files . . . 5

3.2.1 FileReader . . . 5

3.2.2 BufferedReader . . . 6

3.2.3 Scanner . . . 6

3.3 Worked Example: Save and Load Student Grades . . . 6

4 Binary Files 7

4.1 Writing to Binary Files . . . 7

4.1.1 FileOutputStream . . . 7

4.1.2 BufferedOutputStream . . . 7

4.1.3 DataOutputStream . . . 8

4.2 Reading from Binary Files . . . 8

4.2.1 FileInputStream . . . 8

4.2.2 BufferedInputStream . . . 8

4.2.3 DataInputStream . . . 9

5 Text Versus Binary: A Complete Comparison 9

5.1 Same Value, Different Representation 9

6 Exceptions and Resource Management 10

6.1 try-with-resources 10

6.2 End of File and EOFException 10

7 Modern Java File APIs 10

8 More Worked Examples 11

8.1 Example: Count Lines and Words 11

8.2 Example: Copy a Binary File 11

8.3 Example: Save and Restore an Array of Integers 12

9 Common Mistakes to Avoid 12

10 Practice Questions with Answers 13

10.1 Question 1: Which Class Should We Use? 13

10.2 Question 2: What Is Written? 13

10.3 Question 3: Find the Bug 13

10.4 Question 4: Why Use a Buffer? 13

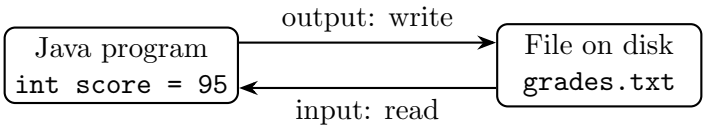
10.5 Question 5: Predict the Output 13

11 Chapter Summary 14

1 What Exactly Is File I/O?

File I/O stands for **File Input and Output**. In simple terms, input means reading data from a file into our Java program, while output means writing data from our Java program into a file. We already know how to print information to the console using `System.out.println()`. File output is similar, except the information is saved somewhere so it can still exist after the program ends.

For example, let's say we create a program that stores student grades. If we keep the grades only in an array, they disappear when the program stops. However, if we write them into a file, we can read them again the next time we run the program. This is the main reason file I/O is useful: **persistent storage**.



The two main categories we will study are **text files** and **binary files**. The basic distinction is how the data is represented and interpreted. Text files contain characters encoded into bytes, while binary files can contain arbitrary byte sequences that are not necessarily meant to be read as text.

Source note: Your original CMSC132 notes list the principal text and binary stream classes and recommend matching writers with readers. The detailed explanations, examples, diagrams, and practice problems below expand that outline. A few simplified statements have been corrected where Java behaves differently.

2 Files, Paths, and Streams

Before we start writing or reading, we need to understand what Java is actually working with. A **path** identifies the location of a file, while a **stream** is a way to transfer data between our program and some source or destination. The source could be a file, but streams can also work with network connections and other data sources.

2.1 Relative and Absolute Paths

A relative path is interpreted from the program's working directory. An absolute path identifies a location from the root of the file system.

```
import java.nio.file.Path;

Path relative = Path.of("data", "grades.txt");
Path absolute = Path.of("/home/student/data/grades.txt");

System.out.println(relative.toAbsolutePath());
```

If your file is not being found, one of the first things to check is the working directory. The file may exist, but not where Java is looking for it. Note that Windows uses different absolute path syntax, such as `C:\\Users\\...` in a Java string.

2.2 The File and Path Classes

The `File` class represents a file or directory pathname. It does *not* automatically open the file or read its contents. The newer `Path` and `Files` APIs are generally more convenient for many tasks.

```
import java.io.File;
import java.nio.file.*;

File file = new File("grades.txt");
System.out.println(file.exists());
System.out.println(file.length()); // Size in bytes

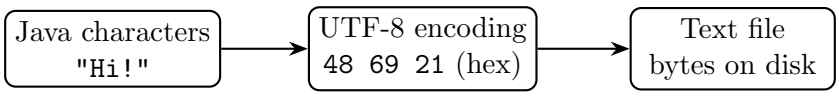
Path path = Path.of("grades.txt");
System.out.println(Files.exists(path));
System.out.println(Files.isRegularFile(path));
```

An important distinction: `File` is about a pathname and metadata. Classes like `FileReader` or `FileInputStream` actually read data. Also, `File.exists()` only checks the state at that moment; it does not guarantee a later read will succeed.

3 Text Files

A text file stores characters using an encoding, such as UTF-8. When we open a text file in an editor, we see letters, digits, punctuation, and newlines. But the disk still stores bytes. A character encoding tells Java how to convert between those bytes and characters.

For example, the text `123` consists of three characters. In UTF-8, the digits are encoded as three bytes, rather than the four bytes used by a Java `int` in a `DataOutputStream`.



3.1 Writing to Text Files

3.1.1 FileWriter

`FileWriter` writes character data to a file. By default, opening a file for writing replaces its existing contents. You can request append mode if you want to keep the existing data.

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteExample {
    public static void main(String[] args) throws IOException {
        try (FileWriter writer = new FileWriter("hello.txt")) {
            writer.write("Hello CMSC132!\n");
            writer.write("This is a second line.\n");
        }
    }
}
```

The `try (...)` syntax is called **try-with-resources**. Java automatically closes the writer when the block finishes, even if an exception occurs. This matters because closing a writer flushes any buffered output and releases the underlying file resource.

Appending instead of replacing:

```
try (FileWriter writer = new FileWriter("hello.txt", true)) {
    writer.write("One more line!\n");
}
```

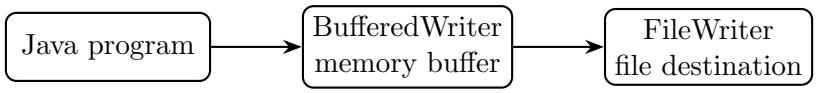
The `true` means append mode. Without it, the file is normally truncated when opened.

3.1.2 BufferedWriter

A `BufferedWriter` wraps another writer and keeps characters in an in-memory buffer before sending them to the underlying destination. This can reduce the number of expensive write operations. It also provides the convenient `newLine()` method.

```
import java.io.*;

try (BufferedWriter writer =
        new BufferedWriter(new FileWriter("students.txt"))) {
    writer.write("Alice,95");
    writer.newLine();
    writer.write("Bob,88");
    writer.newLine();
}
```



Calling `flush()` asks the writer to push buffered characters to the underlying writer, but it does not guarantee that the operating system has permanently committed the data to physical storage. Closing the writer also flushes it.

3.1.3 PrintWriter

`PrintWriter` is convenient because it supports `print()`, `println()`, and `printf()`, just like `System.out`. This is useful when we want to format data.

```
import java.io.*;

try (PrintWriter out = new PrintWriter("report.txt")) {
    out.println("Student Report");
    out.printf("Name: %s%n", "Alice");
    out.printf("Grade: %.2f%n", 95.5);
}
```

One subtle point: many `PrintWriter` methods do *not* throw an `IOException` when a write fails. Instead, you can inspect `checkError()`. For code where error reporting is important, keep that behavior in mind.

3.2 Reading from Text Files

3.2.1 FileReader

`FileReader` reads characters from a file. Its `read()` method returns the next character as an integer, or `-1` when the end of the file is reached.

```
import java.io.*;

try (FileReader reader = new FileReader("hello.txt")) {
    int ch;
    while ((ch = reader.read()) != -1) {
        System.out.print((char) ch);
    }
}
```

The important part is that `-1` is the end-of-file signal, not a character that should be printed. A `FileReader` can also read multiple characters into an array, so it is not limited to single-character operations.

3.2.2 BufferedReader

`BufferedReader` is often the most convenient choice when we want to read a text file **one line at a time**. It uses buffering and offers `readLine()`.

```
import java.io.*;

try (BufferedReader reader =
        new BufferedReader(new FileReader("students.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
}
```

Notice that `readLine()` returns `null` at the end of the file. This differs from `read()`, which returns `-1`. Also, `readLine()` does not include the newline characters in the returned string.

3.2.3 Scanner

`Scanner` is useful when we want to read tokens and parse numbers or other primitive values directly. It is convenient, although usually slower than a simple buffered line reader for large input.

```
import java.io.*;
import java.util.Scanner;

try (Scanner sc = new Scanner(new File("scores.txt"))) {
    while (sc.hasNextInt()) {
        int score = sc.nextInt();
        System.out.println(score);
    }
}
```

If the file contains `95 88 76`, the loop prints those three numbers. However, if the next token is not an integer, `hasNextInt()` returns false. For mixed input, decide carefully whether to use `next()`, `nextInt()`, or `nextLine()`.

3.3 Worked Example: Save and Load Student Grades

Let's put writing and reading together. We will store each student's name and grade on one line, separated by a comma. This is a simplified CSV-style format.

```
import java.io.*;

public class GradeFiles {
    public static void main(String[] args) throws IOException {
        try (PrintWriter out = new PrintWriter("grades.txt")) {
            out.println("Alice,95");
            out.println("Bob,88");
            out.println("Chris,91");
        }

        try (BufferedReader in =
                new BufferedReader(new FileReader("grades.txt"))) {
            String line;
```

```
            while ((line = in.readLine()) != null) {
                String[] parts = line.split(",");
                String name = parts[0];
                int grade = Integer.parseInt(parts[1]);
                System.out.println(name + " scored " + grade);
            }
        }
    }
}
```

Output:

```
Alice scored 95
Bob scored 88
Chris scored 91
```

This works for the simple format we chose. A production CSV parser would also need to handle quoted commas, missing values, and malformed lines.

4 Binary Files

Binary files store sequences of bytes without requiring those bytes to represent readable text. Images, compressed files, audio, and many custom file formats are binary. Binary files are *not automatically* smaller or faster than text files in every situation; that depends on the format, encoding, and operations. Their main advantage is that we can store data in a precise byte-oriented representation.

4.1 Writing to Binary Files

4.1.1 FileOutputStream

`FileOutputStream` writes raw bytes. Its `write(int)` method writes the lowest eight bits of the integer argument. It does not automatically encode a Java `int` as four bytes.

```
import java.io.*;

try (FileOutputStream out = new FileOutputStream("bytes.bin")) {
    out.write(65);              // One byte: 0x41
    out.write(66);              // One byte: 0x42
    out.write(new byte[]{1, 2, 3});
}
```

This creates five bytes: `41 42 01 02 03` in hexadecimal. The first two happen to correspond to the letters A and B in ASCII/UTF-8, but the file is still just a sequence of bytes.

4.1.2 BufferedOutputStream

`BufferedOutputStream` wraps a byte output stream and accumulates bytes in memory before forwarding them. This is useful when writing many small pieces of data.

```
try (BufferedOutputStream out = new BufferedOutputStream(
        new FileOutputStream("data.bin"))) {
    for (int i = 0; i < 1000; i++) {
        out.write(i); // Writes only the low 8 bits
    }
}
```

Note that this does *not* store the full integers from 0 through 999. Values above 255 are truncated to one byte. To store full primitive values, use a `DataOutputStream`.

4.1.3 DataOutputStream

`DataOutputStream` lets us write Java primitive values in a defined binary format. For example, `writeInt()` writes four bytes, `writeDouble()` writes eight bytes, and `writeBoolean()` writes one byte.

```
import java.io.*;

try (DataOutputStream out = new DataOutputStream(
        new BufferedOutputStream(new FileOutputStream("student.bin")))) {
    out.writeInt(132);
    out.writeDouble(95.5);
    out.writeBoolean(true);
    out.writeUTF("Alice");
}
```



The drawing shows the logical order of fields, not a scale drawing of their byte lengths. `writeUTF()` uses a length prefix and Java's modified UTF-8 encoding, rather than writing a raw standard UTF-8 string without metadata.

4.2 Reading from Binary Files

4.2.1 FileInputStream

`FileInputStream` reads raw bytes. Like character readers, `read()` returns an integer from 0 to 255 for a byte, or `-1` at end of file.

```
import java.io.*;

try (FileInputStream in = new FileInputStream("bytes.bin")) {
    int value;
    while ((value = in.read()) != -1) {
        System.out.println(value);
    }
}
```

If the file was written using the earlier example, this prints `65`, `66`, `1`, `2`, and `3`.

4.2.2 BufferedInputStream

`BufferedInputStream` wraps another input stream and reads blocks of bytes into a buffer. Your code can still call `read()` one byte at a time, but the underlying stream may be accessed less often.

```
try (BufferedInputStream in = new BufferedInputStream(
        new FileInputStream("bytes.bin"))) {
    byte[] buffer = new byte[1024];
    int count;
    while ((count = in.read(buffer)) != -1) {
        // Process only buffer[0] through buffer[count - 1]
        System.out.println("Read " + count + " bytes");
```

```
    }
}
```

Always use the returned `count`. The final read may fill only part of the buffer.

4.2.3 DataInputStream

`DataInputStream` reads the same primitive representations that `DataOutputStream` writes. The **order and type must match**. If we write an integer, then a double, then a boolean, we must read them in that same order.

```
import java.io.*;

try (DataInputStream in = new DataInputStream(
        new BufferedInputStream(new FileInputStream("student.bin")))) {
    int course = in.readInt();
    double grade = in.readDouble();
    boolean passed = in.readBoolean();
    String name = in.readUTF();

    System.out.println(course + " " + grade);
    System.out.println(passed + " " + name);
}
```

Why does order matter? The binary file does not automatically tell `DataInputStream` which field comes next. It simply reads the number of bytes required by the method you call. If you call `readDouble()` when the next four bytes are an integer, the data will be interpreted incorrectly, and a later read may throw an `EOFException`.

5 Text Versus Binary: A Complete Comparison

Property	Text files	Binary files
Data representation	Encoded characters	Arbitrary bytes
Human readable	Often yes	Not necessarily
Typical classes	`FileReader`, `BufferedReader`, `PrintWriter`	`FileInputStream`, `DataInputStream`, `DataOutputStream`
Parsing	Convert text to numbers with methods such as `parseInt()`	Read primitive representations using matching methods
Size and speed	Depend on encoding and formatting	Depend on chosen binary format
Best suited for	Logs, configuration, readable reports	Images, raw bytes, custom structured formats

5.1 Same Value, Different Representation

Suppose we want to save the integer `1000`. A text writer could write the characters `1`, `0`, `0`, `0`, taking four bytes in UTF-8. A `DataOutputStream.writeInt(1000)` also uses four bytes, but the bytes are `00 00 03 E8` in hexadecimal (big-endian order). The lengths happen to be equal here, but the meanings of the bytes are different.

6 Exceptions and Resource Management

File operations can fail. The file might not exist, the directory might be missing, or the program might not have permission to access it. Java represents many of these problems using `IOException` and its subclasses.

6.1 try-with-resources

For most file examples, try-with-resources is the preferred way to manage readers and writers.

```
import java.io.*;

public class SafeRead {
    public static void main(String[] args) {
        try (BufferedReader reader =
                new BufferedReader(new FileReader("missing.txt"))) {
            System.out.println(reader.readLine());
        } catch (FileNotFoundException e) {
            System.out.println("Could not find the file.");
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

The more specific `FileNotFoundException` catch comes before the general `IOException` catch. Otherwise the specific catch would be unreachable.

6.2 End of File and EOFException

For ordinary `read()` operations, `-1` signals end of file. For `BufferedReader.readLine()`, `null` signals end of file. However, a `DataInputStream.readInt()` expects exactly four bytes. If it cannot read enough bytes, it throws `EOFException`. These are different ways of reporting that no more complete data is available.

7 Modern Java File APIs

Java also provides `Files` methods for common tasks. These often make short file programs much easier to write. For example:

```
import java.nio.file.*;
import java.nio.charset.StandardCharsets;
import java.io.IOException;
import java.util.List;

public class ModernFiles {
    public static void main(String[] args) throws IOException {
        Path path = Path.of("notes.txt");
        Files.writeString(path, "Hello from CMSC132!\n",
                StandardCharsets.UTF_8);

        String content = Files.readString(path,
                StandardCharsets.UTF_8);
        System.out.println(content);

        List<String> lines = Files.readAllLines(path,
                StandardCharsets.UTF_8);
        System.out.println(lines.size());
```

```
    }
}
```

`readString()` and `readAllLines()` are convenient when the file is small enough to fit comfortably in memory. For very large files, streaming through a `BufferedReader` is generally more appropriate.

8 More Worked Examples

8.1 Example: Count Lines and Words

Suppose we have a text file and want to count how many lines and whitespace-separated words it contains.

```
import java.io.*;

public class WordCount {
    public static void main(String[] args) throws IOException {
        int lines = 0;
        int words = 0;

        try (BufferedReader in =
                new BufferedReader(new FileReader("notes.txt"))) {
            String line;
            while ((line = in.readLine()) != null) {
                lines++;
                String trimmed = line.trim();
                if (!trimmed.isEmpty()) {
                    words += trimmed.split("\\s+").length;
                }
            }
        }
        System.out.println("Lines: " + lines);
        System.out.println("Words: " + words);
    }
}
```

The regular expression "
textbackslash
textbackslash s+" means one or more whitespace characters. We check for empty lines because splitting an empty string can otherwise produce an unexpected result.

8.2 Example: Copy a Binary File

When copying a file, we should not assume that its contents are text. Reading raw bytes allows us to copy an image or any other binary file without corrupting it.

```
import java.io.*;

public class CopyFile {
    public static void main(String[] args) throws IOException {
        try (InputStream in = new BufferedInputStream(
                    new FileInputStream("photo.jpg"));
             OutputStream out = new BufferedOutputStream(
                    new FileOutputStream("photo-copy.jpg"))) {
            byte[] buffer = new byte[8192];
            int count;
            while ((count = in.read(buffer)) != -1) {
                out.write(buffer, 0, count);
            }
```

```
        }
    }
}
```

This uses the general `InputStream` and `OutputStream` reference types. Their actual objects are buffered file streams. It is a nice example of polymorphism working with file I/O.

8.3 Example: Save and Restore an Array of Integers

Let's say we want to store an array of scores in a binary file. We will first write the array length, then each element. When reading, the first integer tells us how many more integers to expect.

```
import java.io.*;
import java.util.Arrays;

public class BinaryArray {
    public static void main(String[] args) throws IOException {
        int[] scores = {90, 85, 100, 77};

        try (DataOutputStream out = new DataOutputStream(
                new FileOutputStream("scores.bin"))) {
            out.writeInt(scores.length);
            for (int score : scores) {
                out.writeInt(score);
            }
        }

        int[] loaded;
        try (DataInputStream in = new DataInputStream(
                new FileInputStream("scores.bin"))) {
            int length = in.readInt();
            if (length < 0 || length > 1_000_000) {
                throw new IOException("Invalid array length");
            }
            loaded = new int[length];
            for (int i = 0; i < length; i++) {
                loaded[i] = in.readInt();
            }
        }
        System.out.println(Arrays.toString(loaded));
    }
}
```

Output: `[90, 85, 100, 77]`. The length check is important if we are reading a file that might be corrupted or untrusted. Otherwise a malicious length could make us allocate an enormous array.

9 Common Mistakes to Avoid

1. **Forgetting to close a stream.** Prefer try-with-resources, which handles closing automatically.
2. **Assuming a relative path means the source-code folder.** Relative paths are resolved from the working directory.
3. **Confusing characters with bytes.** `FileReader` reads characters, while `FileInputStream` reads bytes.
4. **Using the wrong end-of-file test.** `read()` returns `-1`; `readLine()` returns `null`.
5. **Reading binary fields in the wrong order.** The types and order must match the format used during writing.

6. **Forgetting that writing can overwrite a file.** Use append mode when you actually want to preserve previous content.
7. **Ignoring encodings.** For portable text files, explicitly use UTF-8 when appropriate.
8. **Assuming buffering changes the file format.** Buffering improves how data is transferred; it does not turn text into binary or change primitive representations.

10 Practice Questions with Answers

10.1 Question 1: Which Class Should We Use?

We want to read a text file line by line. Which is more convenient: `BufferedReader` or `FileInputStream`?

Answer: `BufferedReader`. It has `readLine()`, which returns a complete line as a string. `FileInputStream` works with raw bytes and does not directly interpret lines of text.

10.2 Question 2: What Is Written?

```
try (FileOutputStream out = new FileOutputStream("x.bin")) {
    out.write(300);
}
```

Answer: Only one byte is written. The low eight bits of 300 are 44, so the file contains the byte `0x2C`. To write the complete four-byte integer 300, use `DataOutputStream.writeInt(300)`.

10.3 Question 3: Find the Bug

```
DataOutputStream out = new DataOutputStream(
        new FileOutputStream("x.bin"));
out.writeInt(10);
out.writeDouble(2.5);
out.close();

DataInputStream in = new DataInputStream(
        new FileInputStream("x.bin"));
double x = in.readDouble();
int y = in.readInt();
in.close();
```

Answer: The reading order does not match the writing order. We wrote an integer first and then a double. Therefore, we must call `readInt()` first and `readDouble()` second.

10.4 Question 4: Why Use a Buffer?

Answer: Writing one byte or character at a time can cause many calls to an underlying stream. A buffer groups smaller operations into larger transfers, which often improves efficiency. It does not guarantee a speed improvement in every program.

10.5 Question 5: Predict the Output

Suppose `data.txt` contains the following two lines:

```
A B C
D E
```

And we execute:

```
try (BufferedReader in =
        new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(in.readLine());
    System.out.println(in.readLine());
    System.out.println(in.readLine());
}
```

Answer: The program prints `A B C`, then `D E`, then `null`. After the second line there are no more lines to read.

11 Chapter Summary

File I/O allows Java programs to save data and load it later. The most important decision is whether we are working with characters or raw bytes. For text, use classes such as `BufferedReader`, `BufferedWriter`, and `PrintWriter`. For binary data, use `FileInputStream` and `FileOutputStream`, optionally wrapped in buffered streams. If we want to write and read primitive types in a defined binary format, `DataOutputStream` and `DataInputStream` are useful, but the order and type of every field must match.

Finally, remember to handle exceptions and close your resources. The best default is usually try-with-resources. Once you understand these ideas, most introductory Java file I/O problems become a matter of selecting the right stream, deciding on a file format, and reading or writing in a consistent order.